



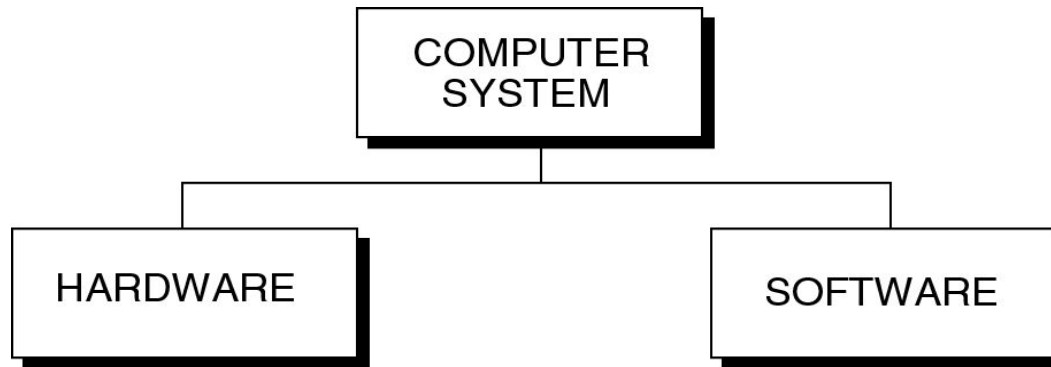
C프로그래밍 (CSE2003)

- 1. Introduction to the C Language**
 - 2. Structure of a C Program**
-

1-1 Computer Systems

- **Computer Systems**

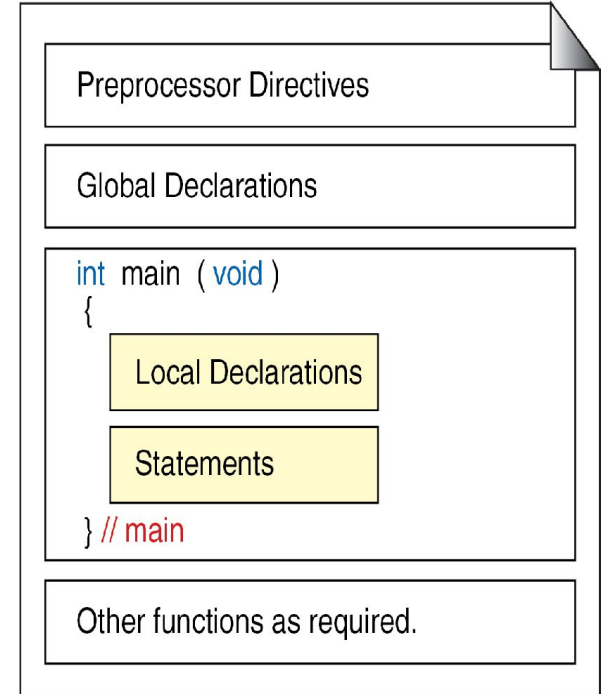
- Data를 다루는 명령어들을 가진 머신(machine)
- System은 두개의 부분으로 구성: hardware and software



- hardware : 컴퓨터를 구성하는 물리적인 장치(device)를 의미: 예) 키보드, CPU, 메모리, 등등
- software : 컴퓨터가 수행할 작업을 지시하는 명령어들의 집합 (프로그램, 이것을 작성하는 작업=프로그래밍), 소프트웨어는 응용 소프트웨어와 시스템 소프트웨어로 나누며, 시스템 소프트웨어는 하드웨어를 작동시키는 기본 소프트웨어 (대표적 예: OS (UNIX,LINUX))

An Overview of C

- C는 Dennis Ritchie가 Unix 운영체제에서 사용되는 DEC PDP-11에서 개발함(1970s)
- C는 구조화된 프로그래밍 언어
 - 어떤 machine 에서도 사용 가능
 - 융통성 있는 다양한 제어 구조와 자료구조, 연산자 가짐
 - 고급언어와 어셈블리 언어의 장점만을 취한 중간수준의 언어
- Compiling a C program
 - Creating your program → file_name.c
 - Compiling your program → many names.o files
 - Linking your program with whatever functions are needed from the library
→ filename.exe file

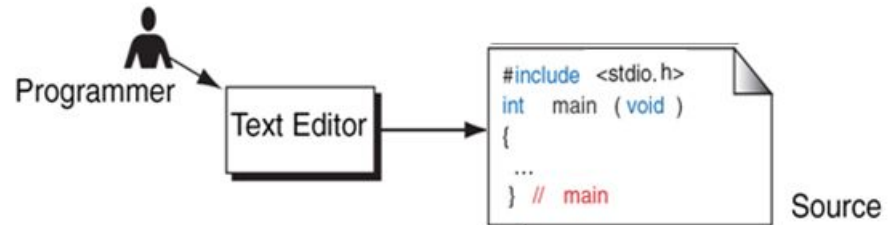


C Program 기본 구조

C program Execution Procedure

C Program 실행 과정

- 1) Edit (Write Code)
- 2) Compile
- 3) Link & Load
- 4) Execute

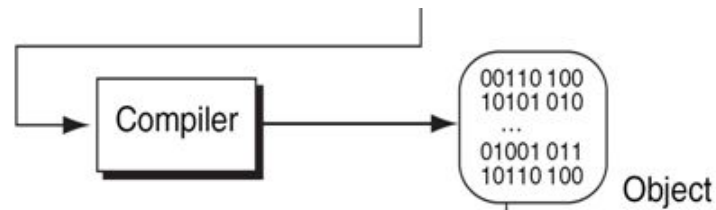


▣ 텍스트 에디터(Notepad)를 사용하여 소스코드를 작성후 저장

C program Execution Procedure

C Program 실행 과정

- 1) Edit (Write Code)
- 2) Compile
- 3) Link & Load
- 4) Execute



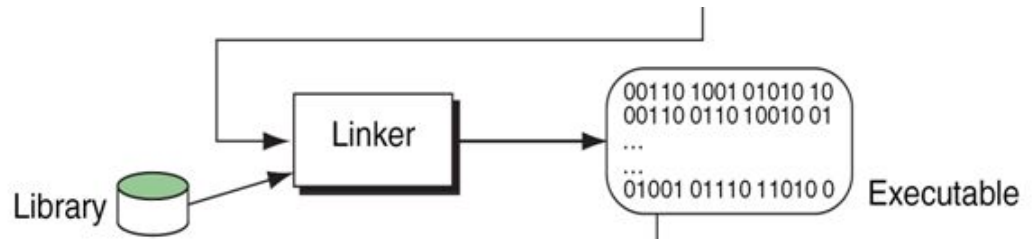
▣ 디스크에 저장된 소스파일은 기계어로 번역되어야 하는데 이 과정을 컴파일링 이라 부르며 C컴파일러는 **preprocessor** 와 **translator** 두 가지 프로그램으로 나뉨

- ▣ **Preprocessor** : 기계어 번역을 위한 준비 수행
- ▣ **Translator** : 준비된 프로그램을 기계어로 번역하여 **object** 모듈 (실행 가능한 완전한 프로그램은 아닌 기계어 모듈)을 만듦

C program Execution Procedure

C Program 실행 과정

- 1) Edit (Write Code)
- 2) Compile
- 3) Link & Load
- 4) Execute



▣ Linker는 번역된 object 모듈들과 library 함수들을 연결하여 실행 가능한 프로그램을 만듦

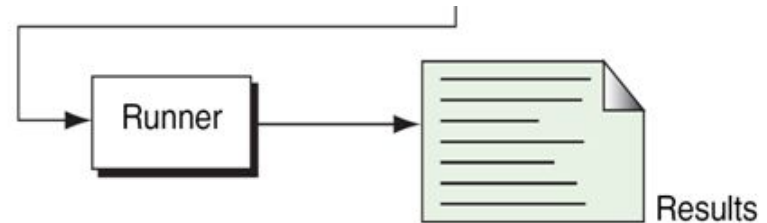
C program Execution Procedure

C Program 실행 과정

- 1) Edit (Write Code)
- 2) Compile
- 3) Link & Load
- 4) Execute

보조기억장치에 저장된 실행 가능한 프로그램을 실행하기 위해서는 프로그램을 메인 메모리에 올리고 실행을 할 운영체제 명령어가 필요 (ex. run)

- Loader는 프로그램을 주 메모리로 옮겨주며, 프로그램이 시스템 컨트롤을 넘겨받게 되면 실행은 시작



Identifier(식별자)

▪ Identifier

- Identifier는 데이터나 객체에 부여하는 이름을 뜻함

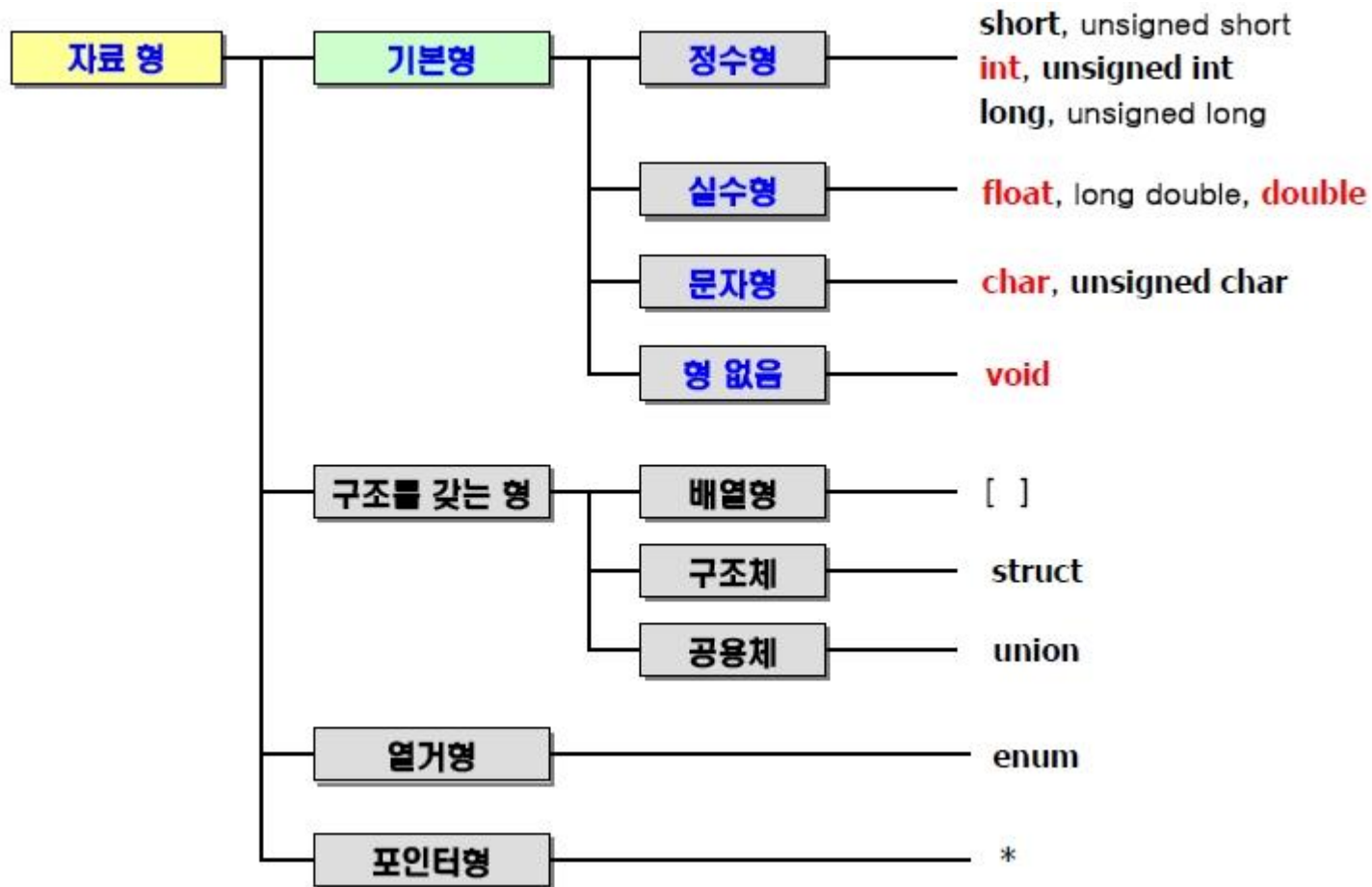
▪ Identifier의 규칙

- 첫 글자는 영문자나 언더스코어('_') 로 시작
- 영문자, 숫자, 언더스코어로만 구성되어야
- 처음 63글자만 구별 가능
- Keyword는 사용할 수 없음 예) double, int, switch, if, else, return, static 등

▪ Identifier의 예

Valid Names		Invalid Names	
a	// Valid but poor style	\$sum	// \$ is illegal
student_name		2names	// First char digit
_aSystemName		sum-salary	// Contains hyphen
_Bool	// Boolean System id	stdnt Nmbr	// Contains spaces
INT_MIN	//System Defined Value	int	// Keyword

Types(자료형)



Types(자료형)

자료형		크기 (byte)	수의 표현 범위
char	char	1	$-2^7 \sim 2^7 - 1$ (-128 ~ 127)
	signed char		
	unsigned char		$0 \sim 2^8 - 1$ (0~255)
int	short int	2	$-2^{15} \sim 2^{15} - 1$ (-32,768 ~ 32,767)
	unsigned short int		$0 \sim 2^{16}$ (0 ~ 65,535)
	int	4	$-2^{31} \sim 2^{31} - 1$ (-2,147,483,648 ~ 2,147,483,647)
	unsigned int		$0 \sim 2^{32} - 1$ (0 ~ 4,294,967,295)
long	long int	4	$-2^{31} \sim 2^{31} - 1$ (-2,147,483,648 ~ 2,147,483,647)
	unsigned long int		$0 \sim 2^{32} - 1$ (0 ~ 4,294,967,295)
float	float	4	$-10^{128} \sim 10^{127}$: 소수 6자리 표현
double	double	8	$-10^{128} \sim 10^{127}$: 소수 15자리 표현
	long double	8 또는 그 이상	차이를 많이 보임 : double의 정밀도와 같거나 크다.

Types(자료형)

▪ Boolean Type(부울형)

- Boolean Type은 true와 false 오직 2가지 값으로 나타냄
- Boolean Type이 규정되기 전에도 integer값을 이용해 표현
 - True ▫ 0 이외의 수 (positive or negative)
 - False ▫ 0 (zero)

▪ Character Type(문자형)

- 1byte로 저장되며 하나의 문자 저장
 - 저장할 수 있는 문자는 alphabet문자에 한정된 것이 아니라 숫자와 기타기호도 포함
 - char형은 0~255의 값을 갖는 작은 int형으로 해석가능
- 그래서 C에서는 char형을 자주 int형과 같이 취급함
- 예) char형 변수는 내부적으로는 정수형이므로 산술연산이 가능
 - 다음의 경우 문자 'A'의 다음 두 번째 문자인 'C'가 출력

```
char c = 'A';           // c = 65
printf("%c", c+2);      // c+2 = 67 => 'C'
```

Types(자료형)

▪ Integer Type(정수형)

- int 형은 정수를 저장
 - short int, int, long int, long long int 로 구분됨
 - 각각의 integer size 들은 signed 혹은 unsigned 될 수 있음

▪ Real Type(실수형)

- float 형은 실수를 저장
- real type 은 float, double, long double로만 구분됨

예약어

- C언어에서 특별한 의미로 사용되는 단어
- 프로그램 내에서 재정의 되거나 다른 용도로 사용되어서는 안됨

auto	default	float	register	switch
break	do	for	return	typedef
cast	double	goto	signed	union
char	else	if	sizeof	unsigned
const	enum	int	static	void
continue	extern	long	struct	volatile
while				

Variables(변수) And Constants(상수)

■ Variables(변수)

- 값을 저장할 수 있는 기억장소에 이름을 붙인 것으로 선언된 후 쓰여야 함
- 변수를 선언할 때는 변수의 type과 변수의 이름을 나란히 써줌
- 변수의 선언은 함수의 첫 부분에서만 가능

```
bool    fact;
short   maxItems;           // Word separator: Capital
long    long national_debt; // Word separator: underscore
float    payRate;           // Word separator: Capital
double  tax;
float    complex voltage;
char     code, kind;         // Poor style—see text
int      a, b;               // Poor style—see text
```

■ variable initialization(변수의 초기화)

- 변수를 초기화하지 않으면 어떤 값이 들어있는지 알 수 없음
초기화 이전에 저장되어 있는 값을 쓰레기 값(garbage)이라 함

■ Constants(상수)

- 프로그램이 실행되는 동안에 변하지 않는 데이터 값

```
const float pi = 3.14159
```

문자 (Characters and Char)

- 모든 정수적형의 변수 (integral type variables) 는 문자를 표현하는 데 사용될 수 있음 (both **char** and **int** are used)
- **'a', '+'와 같은 상수는 char 형이 아니라 int 형임 (It is type int!)**
 - C에는 char 형 상수가 없음

char 자료형

- char 형 변수는 문자와 정수 값을 저장하는 데 사용됨
- 보통 각 char는 메모리의 **1 바이트**에 저장됨
 - 256개 (why?) 의 값을 저장할 수 있음
 - 이 값들 중 일부분만이 실제 인쇄 문자임
(소문자, 대문자, 숫자, 마침표, 특수문자 등)
 - 공백(blank), 탭, 개행 문자 같은 여백 (white space char.) 도 있음
- 대부분의 컴퓨터는 **ASCII**나 **EBCDIC** 문자 코드 사용 (We go with ASCII).

ASCII 코드

- **ASCII 코드에서 문자 상수와 대응되는 정수 값 (No particular rule)**

문자 상수	'a'	'b'	'c'	...	'z'
대응하는 값	97	98	99	...	122
문자 상수	'A'	'B'	'C'	...	'Z'
대응하는 값	65	66	67	...	90
문자 상수	'0'	'1'	'2'	...	'9'
대응하는 값	48	49	50	...	57
문자 상수	'&'	'*'	'+'		
대응하는 값	38	42	43		

ASCII 코드

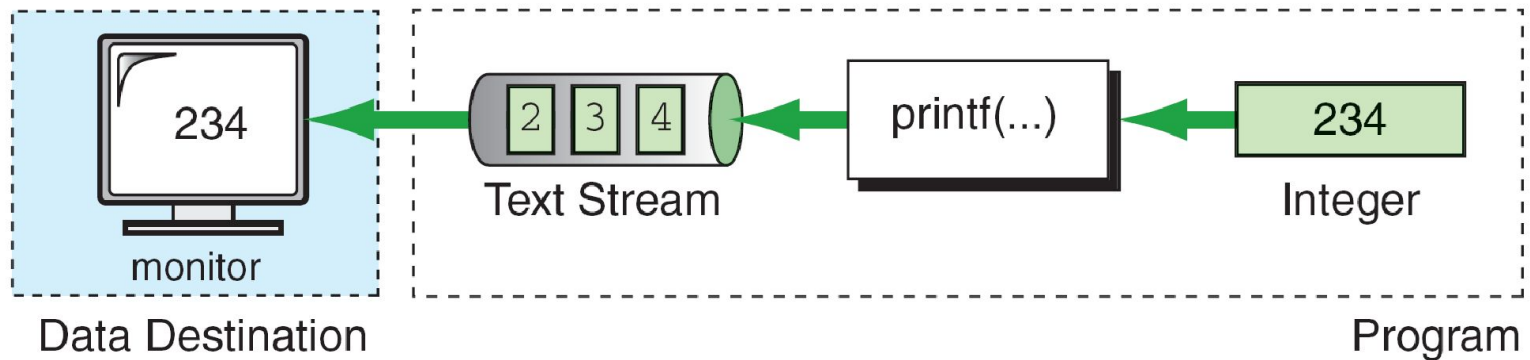
- 주의
 - 숫자를 표현하는 문자 상수의 코드 값과 실제 그 숫자 값은 특별한 관계가 없음
 - 즉, '2'의 값은 2가 아님!!
 - 'a', 'b', 'c' 등의 코드 값은 순서적임 (remember this)
 - 이것은 문자와 단어를 사전적 순서로 정렬할 때 편리함

탈출 기법

- 인쇄할 수 없는 문자는 탈출 기법을 사용하여 표현함
 - 예를 들어, 수평 탭 문자는 문자 상수와 문자열에서 `\t` (backslash with t) 로 표현됨
 - `\t`가 `\`와 `t` 두 문자로 기술되지만, 이것은 한 문자임
 - `\` is called escape character. (backslash)
- 프로그램 내에서 특별한 의미를 갖는 문자들이 본래의 의미를 갖기 위해서도 탈출 기법을 사용해야 함

출력함수 : printf

- C에서 출력을 하는 방법은 여러가지가 있지만 가장 쉽게 쓸 수 있는 것은 printf(print formatted)이다.



- “ ” 안에 출력할 내용을 적는다.
- 변수의 값을 출력할 때는 “ ” 안의 해당 위치에 형식문자(%d, %c, etc..)를 적고, “ ” 뒤에 해당하는 변수를 순서대로 적는다.

```
printf (" ... %d ... %f ...", int형변수, float형변수);
```

출력 함수 : printf

▪ int형 변수 값의 출력 형식

- %d 사이에 폭을 나타내는 정수를 기술
- 필드 폭을 지정하면 자동적으로 우측정렬이 된다. 좌측정렬을 위해서는 필드 폭 앞에 '-'를 붙임

전체 폭은 5이며,
정렬은 기본적으로 우측정렬

```
printf("%5d", 30);
```



			3	0
--	--	--	---	---

전체 폭은 5이며,
정렬은 기본적으로 우측정렬,
+ 의미는 값의 부호 출력

```
printf("%+5d", 30);
```



		+	3	0
--	--	---	---	---

▪ float형 변수 값의 출력 형식

- 필드 폭을 지정하려면 %f 사이에 폭을 기술
- %10f 라고 기술하면
전체 폭은 10, 소수점 이하 자리수 폭은 기본 6으로 지정
- %10.5f 이면
전체 폭은 10, 5는 소수점 이하 자리수의 폭을 의미
- 지정한 전체 폭이 출력 값의 전체 폭보다 작으면, 지정된 작은 폭은 무시하고 원래의 출력 값의 폭으로 출력

전체 폭은 8이며, 변환문자 f는
소수점 이하 6자리가 기본

```
printf("%f",
```

```
3.14);
```



3	.	1	0	0	0	0	0
---	---	---	---	---	---	---	---

전체 폭은 5이며, 소수점
이하 2자리, 정렬은 -
부호에 의해 좌측정렬

```
printf("%-5.2f",
```

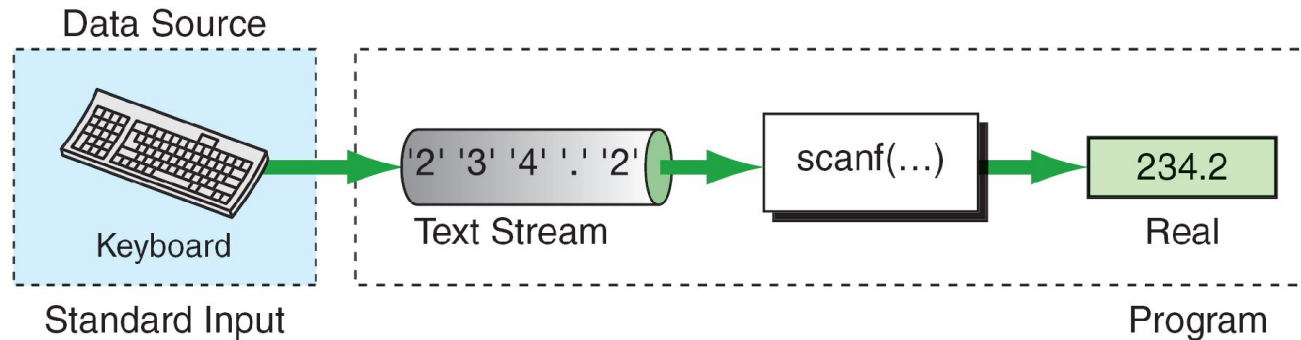
```
3.14);
```



3	.	1	0	
---	---	---	---	--

입력 함수: scanf

- C에서 입력을 받는 방법 중 가장 쉽게 쓸 수 있는 것은 `scanf()`이다.



- `scanf()`는 도스창인 표준 입력으로부터 여러 종류의 자료 값을 입력 받을 수 있음
- 사용법은 `printf()`와 유사
- 단, 입력 값을 저장할 변수의 이름 앞에 '&'를 반드시 붙여줘야 한다.
(&은 해당 변수의 주소를 의미하는 연산자)

```
scanf(" %c %f", &age, &weight);
```

C program Execution Procedure

Step1. Writing and Edit (hello.c)

```
#include <stdio.h>

int main(void)
{
    printf("Hello C\n");
    return 0;
}
```

gcc는 C언어의 compile 명령어로서 compile시 옵션을 주지 않을 경우 default로 a.out이라는 실행 파일이 생성 된다.

Step2 & Step3. Compiling and Linking Programs

```
gr120090364@cspro:~$ vi hello.c
gr120090364@cspro:~$ gcc hello.c
```

Step4. Execution

```
gr120090364@cspro:~$ ./a.out
Hello C
```

C program Execution Procedure

Step1. Writing and Edit (hello.c)

```
#include <stdio.h>

int main(void)
{
    printf("Hello C\n");

    return 0;
}
```

gcc에서 -o 옵션을 사용하면 실행파일의 이름을 선언할수 있다. 여기서는 실행 파일의 이름을 **hello.out** 으로 지정하였다.

Step2 & Step3. Compiling and Linking Programs

```
gr120090364@cspro:~$ vi hello.c
gr120090364@cspro:~$ gcc -o hello.out hello.c
```

Step4. Execution

```
gr120090364@cspro:~$ ./hello.out
Hello C
```


Example Program

```
#include <stdio.h>
```

```
int main(void)
{
```

```
    char ch;
    int integer;
    float real;
```

```
    printf("Input one character, one integer, and one real number : ");
    scanf("%c %d %f", &ch, &integer, &real);
    printf("%c %d %f\n", ch, integer, real);
```

```
    return 0;
}
```

Ch, integer, real 는 identifier 이고

Ch는 문자형 변수, integer는 정수형 변수, real는 실수형 변수이다.

scanf() 함수를 통해 값을 입력 받는다.

입력받을 때는 변수이름 앞에 &를 붙여준다.

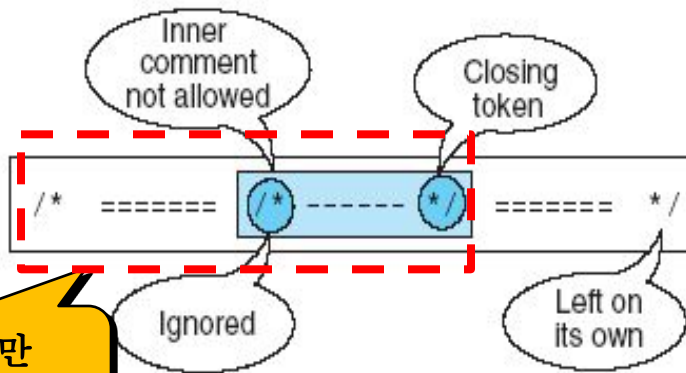
printf() 함수를 통해 값을 출력한다.

값을 입력받고 출력할 때 type에 따라 다른 형식문자(%c, %d, %f)를 사용한다.

```
gr120090364@cspro:~$ vi example-1.c
gr120090364@cspro:~$ gcc -o example-1.out example-1.c
gr120090364@cspro:~$ ./example-1.out
Input one character, one integer, and one real number : d 7 4.23
d 7 4.230000
gr120090364@cspro:~$
```

하나의 scanf () 함수에서 여러 개의 입력을 받을 때 각각의 입력 값은 스페이스로 구분을 하여 입력한다.

C Programs - Comment



이 부분만
주석으로 처리

```
1 /* This is a block comment  
2 that covers two line */  
3  
4 // This is a block comment that covers one line  
5  
6 /* Sogang university /* Computer */ Engineering */  
7  
8
```

주석처리 되지 않은
부분

- comment는 reader가 프로그램을 이해하는데 도움을 줌
- opening token: `/*`
- closing token: `*/`
- 컴파일러는 `/* ..... */` 사이의 모든 내용을 무시
- 컴파일러는 `// ..... 다음의 한 라인의 내용` 을 무시
- 주석 내부에 주석을 중복하여 사용하는 경우 일부는 주석처리가 되지 않는다.

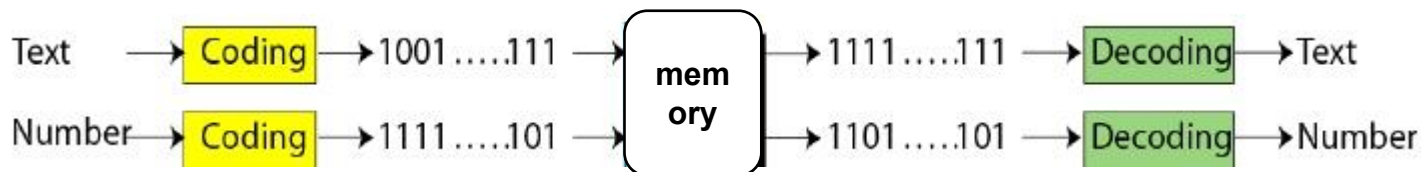
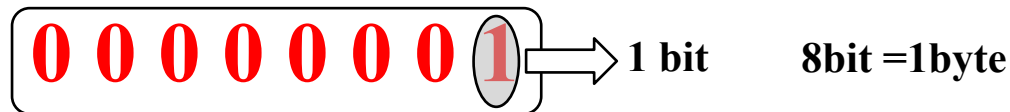
데이터 표현

- 비트(bit)

- 컴퓨터에 저장 될 수 있는(0또는 1) 데이터의 가장 작은 단위

- 비트패턴(bit pattern)

- 서로 다른 타입의 데이터를 표현하기 위해 비트 패턴을 사용하게 된다.
 - 데이터는 컴퓨터에 들어갈 때 코드화되고 사용자에게 표현 될 때 원상 복구된다.

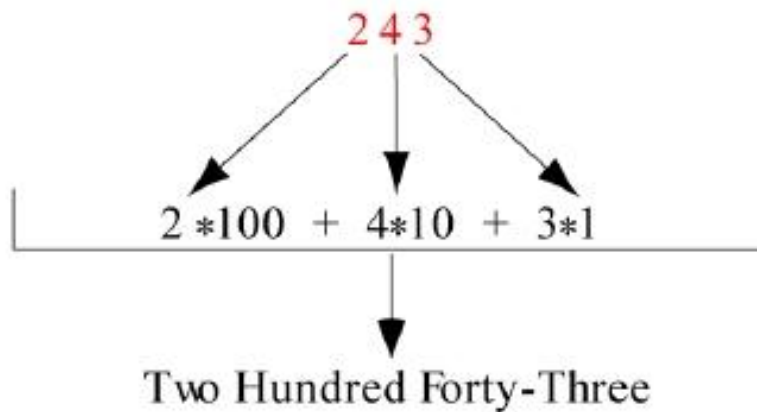


진법과 변환

- 10진법(decimal system)

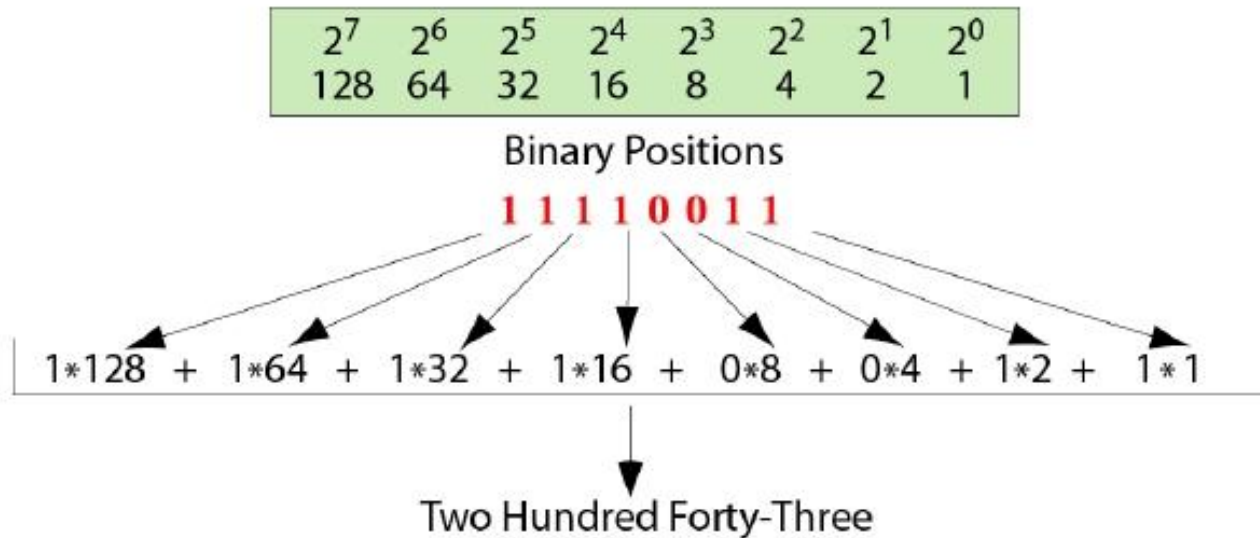
10^4	10^3	10^2	10^1	10^0
10,000	1000	100	10	1

Decimal Positions



진법과 변환(cont'd)

- 2진법(binary system)



진법과 변환(cont'd)

- 8진 표기법(octal notation)
 - . 8에 기초를 두고 8개의 기호를 이용
 - 0,1,2,3,4,5,6,7
 - 각각의 8진수는 3비트로 표현 할 수 있다.
 - 3비트는 8진수 한 개로 표현 될 수 있다.

Bit Pattern	Hexadecimal digit	Bit Pattern	Hexadecimal digit
000	0	100	4
001	1	101	5
010	2	110	6
011	3	111	7

진법과 변환(cont'd)

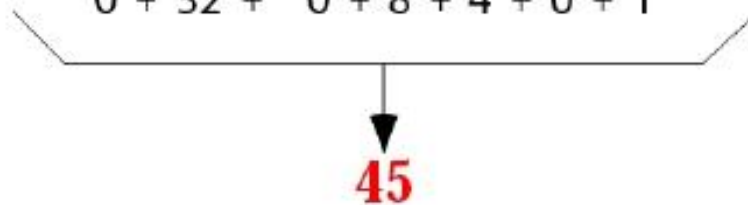
- 16진 표기법(hexadecimal notation)
 - . 16에 기초를 두고 16개의 기호를 이용한다.
 - . 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F
 - . 각각의 16진수는 4비트를 표현 할 수 있다.
 - . 4비트는 16진수 한 개로 표현 될 수 있다.

Bit Pattern	Hexadecimal digit	Bit Pattern	Hexadecimal digit
0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F

진법과 변환(cont'd)

- 2진수 10진 변환

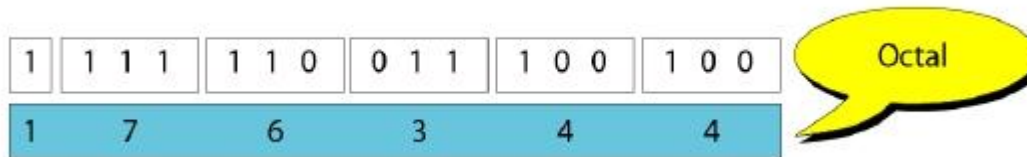
. 2진수에서 각 자리의 숫자에 그 자리의 가중치를 곱한다.

0	1	0	1	1	0	1	binary number
64	32	16	8	4	2	1	position values
							results
0 + 32 + 0 + 8 + 4 + 0 + 1							
							decimal number

진법과 변환(cont'd)

- 8진수의 변환

- 8진수를 비트 패턴으로 전환 할 경우 각각의 8진수를 3비트 해당 값으로 전환하면 된다.



- 8진수 표기법

- 숫자 앞에 0을 더해 준다(예 : 0634)
- 숫자(8)를 8진수 뒤에 표기한다(예: 634₈)

진법과 변환(cont'd)

- 16진수의 변환
 - . 16진수에 대한 비트 패턴 변환은 각각의 16진수를 4비트와 같은 값으로 변환하는 것이다.

1 1 1 1	1 1 0 0	1 1 1 0	0 1 0 0
F	C	E	4

Hexadecimal

- 16진수 표기법
 - . 숫자 앞에 소문자(또는 대문자) x를 추가한다(예: **x**A34)
 - . 숫자(16)를 16진수 뒤에 표기한다. (예: A34₁₆)

실습1